

Sistemas Embebidos I

Otoño 2026

Práctica 7: PWM

Jonathan Hernández Lazcano

Eduardo Gabriel Prado Granados

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Hamid Susen Santiago García

Lunes, 28 de septiembre de 2026

Índice

Índice 1
Examen 1: Duelo de reacción 2
Introducción 2
Descripción de la práctica 2
Objetivo 3
Marco teórico 3
Materiales 5
Esquemático 5
Desarrollo 5
Descripción general del sistema 5
Diagrama de conexiones y montaje 6
Diagrama de bloques: qué corre en cada ISR y qué corre en el bucle 8
Máquina de estados del juego 10
Fragmentos de código explicados 10
1. Configuración de botones, UART e interrupciones 10
2. ISR de UART y buffer circular de recepción 11
3. Transmisión no bloqueante 12
4. Consola: eco, armado de línea y comandos 13
5. Inicio de partida y tiempo aleatorio 15
6. Callback del timer 16
7. ISR de los botones: antirrebote y medición 17
8. Reporte del resultado y bucle principal 19
9. Programa de la Pico de apoyo: puente USB–UART 20
Pruebas de funcionamiento por consola UART 21
Verificación del tiempo de reacción con osciloscopio 25
Primera medición: modo Roll 25
Segunda medición: tiempo de reacción 26
Duración de la pulsación 27
Comparación con la consola UART 28
Conclusiones **28**
Referencias 29

En los sistemas embebidos, los microcontroladores permiten manejar dispositivos con instrucciones programadas, siendo que el software queda vinculado de manera directa con la parte física. Sin embargo, generar señales digitales para que todo funcione bien no es suficiente, en ocasiones es necesario ajustarlas según los componentes que se utilizan.

Una técnica muy utilizada es la modulación del ancho de pulso, o PWM por sus siglas en inglés, la cual regula el comportamiento de distintos dispositivos mediante una señal digital periódica. Al modificar parámetros como la frecuencia y el ciclo de trabajo, es posible variar el brillo de un LED, controlar la velocidad de un motor, o controlar el ángulo de giro en un servomotor.

En esta práctica se hace uso de la Raspberry Pi Pico 2W para controlar cuatro cargas con PWM de manera simultánea, y se integra esta configuración con una consola de comunicación serial. Además de desarrollar el programa de control, se busca observar y comparar las señales generadas con las respuestas reales de los dispositivos, tomando en cuenta tanto los parámetros programados como las limitaciones eléctricas del montaje.

Descripción de la práctica

La práctica consiste en desarrollar un sistema de control basado en PWM utilizando una Raspberry Pi Pico 2W y el SDK de C/C++. El sistema debe permitir la manipulación de cuatro dispositivos mediante comandos enviados desde una consola serial: un LED conectado al GPIO 2, un buzzer pasivo al GPIO 4, un servomotor al GPIO 6 y un motor de corriente directa mediante un MOSFET al GPIO
8.

Cada dispositivo requiere de una configuración particular de frecuencia y ciclo de trabajo según su funcionamiento. Para ello, se establecen los parámetros del periférico PWM del microcontrolador y se implementa el procesamiento de comandos que permiten modificar el comportamiento de las cargas durante la ejecución.

El desarrollo contempla la comprobación del brillo del LED, la generación de diferentes tonos en el buzzer, el ajuste de la posición del servomotor y la regulación de la

velocidad del motor de corriente directa. Asimismo, se realizan mediciones y cálculos para comparar los valores esperados contra los valores obtenidos de manera experimental. Finalmente se verifica el funcionamiento simultáneo de los cuatro dispositivos y la capacidad del programa para responder a nuevos comandos mientras se ejecutan determinadas acciones.

Objetivo

Implementar un sistema de control de cuatro dispositivos de manera simultánea mediante la técnica PWM en una Raspberry Pi PIco 2W, integrando la configuración de los periféricos del microcontrolador con una consola de comunicación serial. Se busca ajustar y evaluar los parámetros de funcionamiento de un LED, un buzzer, un servomotor y un motor de corriente directa, comparando los valores teóricos con las condiciones experimentales y verificando la operación simultánea de las cargas.

Marco teórico

Fundamentos y parámetros del PWM

La modulación por ancho de pulso o PWM, es una técnica utilizada para llevar el control de la cantidad de energía eléctrica que se envía a un dispositivo electrónico mediante la variación rápida de los tiempos de encendido y apagado de una señal digital.

En lugar de variar el voltaje continuo, la señal PWM mantiene un voltaje pero varia el tiempo que permanece activa, simulando una señal analógica. Para entender y configurar una señal PWM, se deben dominar cuatro parámetros, siendo el duty cycle, periodo, frecuencia y voltaje promedio.

Duty cycle, periodo, frecuencia y voltaje promedio

Como parte de los parámetros que se deben modular en una señal PWM, el duty cycle representa el porcentaje de tiempo que la señal está en estado alto dentro de un periodo completo, este se mide en porcentaje.

Por otro lado, el periodo es el tiempo total que tarda la señal en completar un ciclo completo, este se mide en segundos o ms, mientras que la frecuencia es la cantidad

de ciclos completos que la señal realiza en 1 segundo. Se mide en Hertz (Hz) y es el inverso del periodo.

Finalmente, el voltaje promedio es el voltaje resultante que siente el componente conectado, y este se calcula multiplicando el voltaje máximo por el ciclo de trabajo (duty cycle) en decimal.

Aplicaciones de PWM en distintos actuadores

La modulación por ancho de pulso tiene distintas aplicaciones dependiendo de las características del dispositivo controlado y de la manera en que este responde a la señal eléctrica. En un sistema embebido, el PWM puede utilizarse tanto para regular la energía suministrada a una carga como para transmitir información mediante pulsos de duración o frecuencia determinadas.

En un LED, la variación del ciclo de trabajo permite modificar la intensidad luminosa. Un ciclo de trabajo bajo reduce el tiempo durante el cual el LED permanece encendido, mientras que uno alto incrementa dicho tiempo. Sin embargo, el brillo percibido no aumenta de manera lineal con el ciclo de trabajo, debido a las características de la percepción visual humana.

Otra aplicación es la generación de tonos mediante un buzzer pasivo. El buzzer pasivo requiere una señal periódica para producir sonido. En este caso, la frecuencia de la señal determina principalmente el tono generado: al aumentar la frecuencia, se obtiene un sonido más agudo, mientras que al disminuirla, el sonido es más grave. El ciclo de trabajo también puede modificar las características de la señal y afectar el volumen percibido y el timbre del sonido.

En un servomotor convencional, la posición se determina principalmente mediante la duración del pulso de control. Una señal periódica con un periodo aproximado de 20 ms, equivalente a una frecuencia de 50 Hz, puede utilizar pulsos de alrededor de 1 ms, 1.5 ms y 2 ms para indicar distintas posiciones angulares. Los valores exactos dependen del modelo y deben verificarse para evitar que el mecanismo alcance sus límites físicos.

PWM también permite regular la velocidad de un motor DC mediante la conmutación de la alimentación. Al modificar el ciclo de trabajo, cambia la tensión promedio aplicada al motor y, con ello, su comportamiento. No obstante, la velocidad real también depende de factores como la carga mecánica, la resistencia interna, la alimentación y las características del propio motor. Además, puede existir un ciclo de trabajo mínimo necesario para iniciar el movimiento.

Estas aplicaciones muestran que el PWM es una herramienta versátil para el control de dispositivos electrónicos. Aunque la señal puede generarse mediante un mismo periférico del microcontrolador, cada carga requiere parámetros específicos de frecuencia, duración del pulso y ciclo de trabajo según su principio de funcionamiento.

Etapa de potencia y consideraciones eléctricas

La etapa de potencia es el conjunto de componentes electrónicos encargados de suministrar al actuador la energía necesaria para su funcionamiento. Debido a que los microcontroladores normalmente no pueden proporcionar directamente la corriente requerida por motores u otras cargas, se utilizan dispositivos de conmutación como transistores MOSFET, IGBT o circuitos integrados controladores.

Esta etapa recibe la señal PWM y controla el paso de energía desde una fuente de alimentación hacia el actuador. También puede incluir diodos de protección, condensadores, controladores de compuerta y disipadores térmicos. La selección de estos componentes depende de la tensión, la corriente y la potencia requeridas por la carga (Texas Instruments, s. f.).

Para diseñar una etapa de potencia confiable, es necesario considerar los siguientes aspectos:

- Tensión y corriente: los componentes deben soportar los valores nominales y los picos que puedan producirse durante el funcionamiento.
- Frecuencia PWM: influye en las pérdidas de conmutación, el ruido y el comportamiento del actuador.
- Disipación térmica: los transistores generan calor debido a las pérdidas eléctricas, por lo que puede ser necesario utilizar disipadores o sistemas de enfriamiento.

- Protección de cargas inductivas: los motores y solenoides pueden producir sobretensiones al interrumpirse la corriente. Para prevenir daños se emplean diodos de rueda libre u otros circuitos de protección.
- Interferencias electromagnéticas: las conmutaciones rápidas pueden afectar otros circuitos, por lo que es importante diseñar correctamente las conexiones, la puesta a tierra y el filtrado.

Materiales

- 1 LED y su resistencia limitadora, conectados al pin GP16 de la Pico principal.
- · 2 cables USB tipo A a micro-USB, uno por tarjeta.
- Computadora con Visual Studio Code y la extensión *Raspberry Pi Pico*.
- Pico SDK 2.3.1 con la cadena de herramientas arm-none-eabi-gcc 13.2, CMake y Ninja.
- Raspberry Pi Pico o Pico W
- Computadora con Visual Studio Code y la extensión *Raspberry Pi Pico*.
- Pico SDK 2.3.1 con la cadena de herramientas arm-none-eabi-gcc 13.2, CMake y Ninja.
- Protoboard
- Cable USB de datos
- Cables jumper macho-macho
- LED
- Resistencia de 220 Ω a 330 Ω
- Buzzer pasivo
- Servomotor, por ejemplo SG90
- Motor de corriente continua de 5 V
- Transistor MOSFET de canal N
- Diodo de protección

- Fuente externa regulada de 5 V
- Osciloscopio
- Multímetro digital
- Computadora

Esquemático

Desarrollo

Descripción general del sistema

Un solo programa controla cuatro cargas al mismo tiempo, cada una con su sistema está formado por dos programas, cada uno en su propia Raspberry Pi Pico 2. **PICO_PRINCIPAL_EXAMEN**propia señal PWM: un LED en el GPIO 2, un buzzer pasivo en el GPIO 4, un servomotor en contiene toda la lógica del juego: botones, LEDs, temporizador,el GPIO 6 y un motor de corriente directa en el GPIO 8 a través de un MOSFET de canal N. medición del tiempo de reacción y consola de comandos por UART0. **PICO_APOYO_EXAMEN**Todas las cargas se manejan con comandos de texto escritos en una consola serial que se se ejecuta en una segunda tarjeta que no participa en el juego: solocomunica con la tarjeta por la UART0 (GP0 = TX, GP1 = RX) a 115200 baudios en formato reenvía bytes entre el puerto USB de la computadora y la UART de la Pico principal, como un intérprete que traduce entre dos personas que hablan idiomas distintos. 8N1.

El juego se opera desde una consola serial. Al encenderse, envía por UART un mensaje de La idea central de la práctica es que el PWM no es una sola configuración que sirve para bienvenida con la lista de comandos. Una partida inicia con el comando start: el LED rojo todo, sino un mismo "lenguaje" que cada dispositivo interpreta a su manera. Un buen comienza a parpadear con el periodo configurado y, después de un tiempo aleatorio entre 2 ejemplo es una llave de agua que se abre y se cierra muy rápido: al LED y al motor les y 5 segundos, se apaga y se enciende el LED verde. El primer jugador que presiona su importa cuánto tiempo permanece abierta (el ciclo de trabajo, que fija el voltaje promedio), al botón con el LED verde encendido gana y el sistema reporta su tiempo de reacción. Si un buzzer le importa qué tan rápido se abre y se cierra (la frecuencia, que fija el tono) y al jugador presiona antes del verde comete una salida en falso y el punto es para su rival; si servomotor le importa la duración exacta de cada chorro (el ancho del pulso, que fija el nadie presiona durante 5 segundos, la partida termina sin ganador. ángulo). Esta distinción coincide con lo que describe Sufyan (2025): en un servomotor la posición queda codificada por el ancho absoluto del pulso y no por el voltaje promedio,**Tabla 1.** *Comandos de la consola UART implementados.* mientras que en LEDs y motores es el voltaje promedio el que determina el brillo o la

velocidad.

El trabajo se realizó en tres etapas. Primero se programó y probó el sistema en el simuladorstart Inicia una partida si no hay otra en curso Listo o Error: partida en curso Wokwi, donde se verificó la lógica de los comandos y se capturaron las señales con elset periodo <ms> Cambia el periodo de parpadeo del LED Listo o Error: periodo fuera de analizador lógico para revisarlas en Surfer. Después se armó el circuito físico en protoboard rojo (50–1000 ms); solo con el juego rango (50-1000 ms)

y se cargó el mismo programa en la Pico 2W. Por último, se midieron las señales con el detenido osciloscopio y el multímetro para comparar los valores teóricos con los experimentales.

Asignación de pines y slices de PWM

El RP2350 genera PWM con bloques de hardware llamados *slices*. Cada slice tiene un contador de 16 bits, un divisor de reloj y dos canales (A y B). Los dos canales de un slice comparten la misma frecuencia, pero cada uno puede tener su propio ciclo de trabajo. Por eso los pines se eligieron de manera que cada carga quedara en un slice distinto: así el buzzer puede cambiar de frecuencia con cada nota sin alterar los 50 Hz del servomotor ni los 20 kHz del motor. Es como asignar a cada músico su propio metrónomo en lugar de obligarlos a todos a seguir el mismo tempo.

Tabla 1. *Asignación de pines, slices y canales de la Pico 2W.*

Pin	Función	Slice / canal	Configuración
GP0	UART0 TX (consola)	—	GPIO_FUNC_UART
GP1	UART0 RX (consola)	—	GPIO_FUNC_UART
GP2	LED + resistencia de 220 Ω	Slice 1, canal A	GPIO_FUNC_PWM
GP4	Buzzer pasivo	Slice 2, canal A	GPIO_FUNC_PWM
GP6	Señal del servomotor SG90	Slice 3, canal A	GPIO_FUNC_PWM
GP8	Compuerta del MOSFET (motor DC)	Slice 4, canal A	GPIO_FUNC_PWM

El número de slice no se escribió a mano en el programa: se obtiene con pwm_gpio_to_slice_num() y pwm_gpio_to_channel(), de modo que si un pin cambia, el código sigue siendo correcto.

Diagrama de conexiones y etapa de potencia

El LED se conecta en configuración de fuente de corriente: el GPIO 2 entrega corriente a través de una resistencia de 220 Ω hacia el ánodo, y el cátodo va a tierra. El buzzer pasivo se conecta entre el GPIO 4 y tierra; a diferencia de un buzzer activo, no tiene oscilador interno, por lo que solo suena si recibe una señal periódica. El servomotor recibe su señal de control del GPIO 6, pero su alimentación de 5 V viene de la fuente externa, ya que al moverse puede pedir picos de corriente que el regulador de la Pico no debe entregar.

El motor de corriente directa no puede conectarse directamente al GPIO, porque el pin solo entrega unos cuantos miliamperes a 3.3 V. Por eso se usa una etapa de potencia con un MOSFET de canal N en configuración de lado bajo: el motor va entre los +5 V de la fuente externa y el drenador, la fuente del MOSFET va a tierra y la compuerta recibe el PWM del GPIO 8 a través de una resistencia en serie, con una resistencia de 10 kΩ entre compuerta y fuente para mantener el motor apagado mientras la Pico arranca. El MOSFET funciona como una compuerta de esclusa: la Pico solo da la orden de abrir o cerrar con una señal pequeña, y es la fuente de 5 V la que mueve el agua pesada. Como el GPIO solo llega a 3.3 V, el MOSFET debe ser de nivel lógico, es decir, debe conducir por completo con ese voltaje en la compuerta.

En paralelo con el motor se colocó un diodo de rueda libre, con el cátodo hacia +5 V. El motor es una carga inductiva: cuando el MOSFET se apaga, la bobina intenta mantener la corriente y genera un pico de voltaje que podría dañar el transistor. El diodo le da a esa corriente un camino de regreso, como una válvula de alivio en una tubería que se cierra de golpe. Por último, la tierra de la fuente externa y

la tierra de la Pico se unieron, ya que sin una referencia común el MOSFET y el servomotor no interpretarían correctamente los niveles de la señal. La Figura 1 resume todas las conexiones.

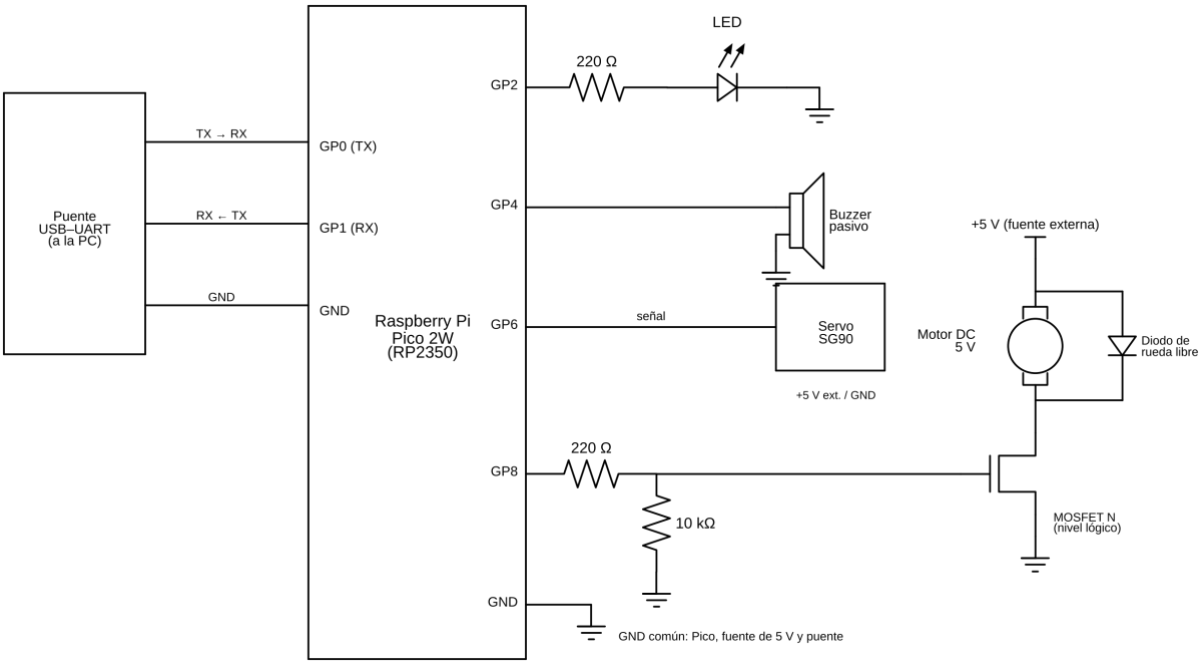


Figura 1. *Diagrama esquemático de las cuatro cargas, la etapa de potencia del motor y la consola UART conectadas a la Pico 2W.*

Cálculo de los parámetros PWM

Cada slice funciona como un corredor que da vueltas a una pista: el contador avanza un paso por cada pulso de reloj dividido y, al llegar al valor TOP (llamado *wrap* en el SDK), vuelve a cero e inicia otra vuelta. La salida permanece en alto mientras el contador es menor que el valor de comparación (*level*) y en bajo el resto de la vuelta. Así, el tamaño de la pista fija la frecuencia y el punto donde se coloca la marca fija el ciclo de trabajo. Las ecuaciones que se usaron son:

$$f_{PWM} = f_{clk} / [div \times (TOP + 1)] \qquad D = level / (TOP + 1) \qquad V_{prom} = V_{alto} \times D$$

donde f_{clk} es el reloj del sistema, que en la Pico 2W es de 150 MHz; el programa no lo supone, sino que lo consulta con clock_get_hz(clk_sys). El divisor puede ir de 1 a 255 + 15/16 y TOP puede llegar a 65 535. Dietrich (2022) explica que la frecuencia debe ser lo bastante alta para que la carga no siga cada transición individual y solo perciba el promedio; Sufyan (2025) agrega que también existe un compromiso con la resolución del ciclo de trabajo, que depende de cuántos pasos tiene el contador en cada periodo. Con esos dos criterios se eligieron los valores de la Tabla 2.

Tabla 2. *Parámetros PWM calculados para cada carga con f_{clk} = 150 MHz.*

Carga	div	TOP	Frecuencia	Resolución	Criterio de diseño
LED	150	999	1 kHz	1000 pasos (0.1 %)	Sin parpadeo visible; 1 paso del contador = 1 µs

Buzzer	10	Variable (2 999 a 65 535)	229 Hz a 5 kHz	—	La frecuencia es el tono; duty fijo de 50 %
Servo	150	19 999	50 Hz (20 ms)	1 µs por paso	level = ancho del pulso en µs
Motor DC	1	7 499	20 kHz	7500 pasos (0.013 %)	Por encima del rango audible

Para el LED y el servomotor se usó el mismo truco: con un divisor de 150, el contador avanza exactamente una vez por microsegundo. En el LED, un TOP de 999 da 1000 pasos por periodo, es decir, 1 kHz con una resolución de 0.1 %. En el servomotor, un TOP de 19 999 da un periodo de 20 ms, y el valor de *level* queda expresado directamente en microsegundos: escribir 1500 produce un pulso de 1.5 ms sin ninguna conversión adicional. Es como usar una regla graduada en las mismas unidades que se quieren medir.

En el buzzer, el divisor se fijó en 10 y lo que cambia con cada nota es TOP, calculado como TOP = round(15 MHz / f) − 1. El divisor se eligió con la nota más grave: con div = 10, la frecuencia mínima alcanzable con TOP = 65 535 es de unos 229 Hz, suficiente para el Do4 (261.63 Hz), que es la nota más baja de las melodías. Como TOP debe ser entero, la frecuencia real difiere un poco de la teórica; la Tabla 3 muestra que el error es menor a 0.003 % en todas las notas, muy por debajo de lo que el oído puede distinguir.

Tabla 3. *Frecuencia teórica, TOP calculado y frecuencia real de algunas notas del buzzer.*

Nota	f teórica (Hz)	TOP	f real (Hz)	Error (%)
Do4	261.63	57 332	261.63	−0.0002
La4	440.00	34 090	440.00	−0.0003
Mi5	659.26	22 752	659.25	−0.0010
La5	880.00	17 044	880.02	+0.0027
Tono máximo	5000.00	2 999	5000.00	0.0000

En el motor se priorizó que la frecuencia quedara por encima del rango audible. Sufyan (2025) señala que los motores DC suelen usar frecuencias de algunos kilohertz hasta decenas de kilohertz para evitar el zumbido y reducir el rizo de par. Con div = 1 y TOP = 7 499 se obtienen 20 kHz y todavía quedan 7500 pasos de resolución. La Figura 3 muestra las cuatro formas de onda teóricas que resultan de estos parámetros; las escalas de tiempo son distintas porque las frecuencias van de 50 Hz a 20 kHz.

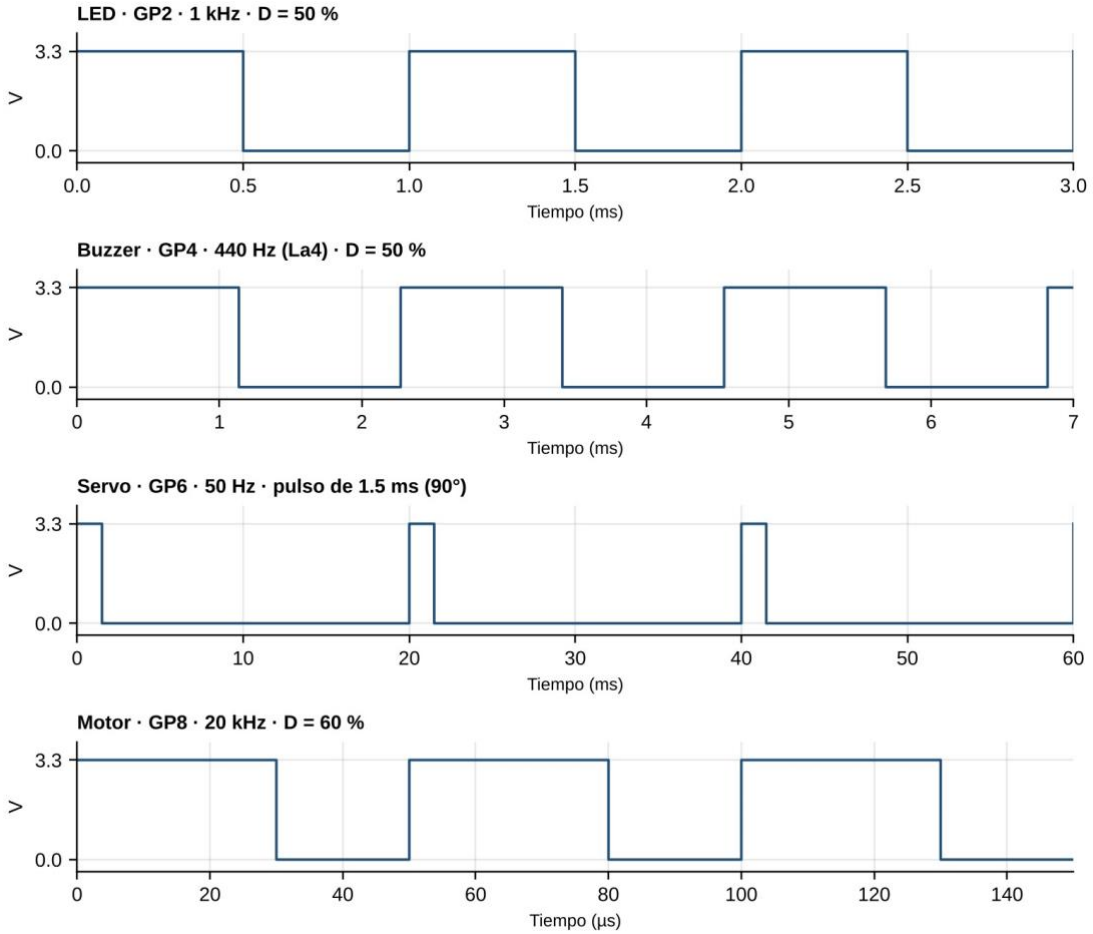


Figura 3. *Formas de onda teóricas de las cuatro señales PWM calculadas con los parámetros de la Tabla 2.*

Configuración del periférico PWM

Toda la configuración se concentra en la función pwm_setup(). Para cada carga se asigna la función PWM al pin, se obtienen su slice y canal, se cargan el divisor y TOP, se arranca con el ciclo de trabajo en cero y, al final, se habilita el slice.

Fragmento 1. Configuración de los cuatro slices en pwm_setup() (main.c).

```
static void pwm_setup(void)
{
    g_clk_hz  = clock_get_hz(clk_sys);   // 150 MHz en la Pico 2W
g_led_div = g_clk_hz / 1000000u;     // 1 paso del contador = 1 us

    // LED: 1 kHz, TOP = 999
    gpio_set_function(PIN_LED,      GPIO_FUNC_PWM);
g_slice_led    =    pwm_gpio_to_slice_num(PIN_LED);
g_chan_led       =    pwm_gpio_to_channel(PIN_LED);
pwm_set_clkdiv(g_slice_led,      (float)g_led_div);
pwm_set_wrap(g_slice_led, LED_TOP);
    pwm_set_chan_level(g_slice_led, g_chan_led, 0);
pwm_set_enabled(g_slice_led, true);

    // Buzzer: div fijo = 10, TOP cambia con cada nota
gpio_set_function(PIN_BUZZER,           GPIO_FUNC_PWM);
g_slice_buz    =    pwm_gpio_to_slice_num(PIN_BUZZER);
g_chan_buz       =    pwm_gpio_to_channel(PIN_BUZZER);
pwm_set_clkdiv(g_slice_buz,            (float)BUZ_DIV);
pwm_set_wrap(g_slice_buz, 65535);
    pwm_set_chan_level(g_slice_buz, g_chan_buz, 0);
pwm_set_enabled(g_slice_buz, true);

    // Servo: 50 Hz, level = ancho del pulso en us
gpio_set_function(PIN_SERVO,         GPIO_FUNC_PWM);
g_slice_srv    =    pwm_gpio_to_slice_num(PIN_SERVO);
g_chan_srv       =    pwm_gpio_to_channel(PIN_SERVO);
pwm_set_clkdiv(g_slice_srv, (float)g_led_div);
    pwm_set_wrap(g_slice_srv, SERVO_TOP);              // 19999
pwm_set_chan_level(g_slice_srv, g_chan_srv, SERVO_CENTRO_US);
pwm_set_enabled(g_slice_srv, true);

    // Motor: 20 kHz, div = 1, TOP = 7499
gpio_set_function(PIN_MOTOR,         GPIO_FUNC_PWM);
g_slice_mot    =    pwm_gpio_to_slice_num(PIN_MOTOR);
g_chan_mot       =    pwm_gpio_to_channel(PIN_MOTOR);
pwm_set_clkdiv(g_slice_mot, 1.0f);
    pwm_set_wrap(g_slice_mot, MOTOR_TOP);              // 7499
pwm_set_chan_level(g_slice_mot, g_chan_mot, 0);    // motor detenido
pwm_set_enabled(g_slice_mot, true);
}
```

Una vez habilitado, cada slice genera su señal por hardware sin intervención del CPU. El programa solo vuelve a tocar un slice cuando un comando cambia su ciclo de trabajo o su frecuencia. Además, los registros de TOP y *level* tienen doble buffer: un valor nuevo se aplica hasta que el contador termina la vuelta en curso, así que un cambio nunca deja un pulso cortado a la mitad.

Control de cada carga

1. LED: brillo, corrección gamma y fade

El comando led <duty> fija el ciclo de trabajo directamente, de 0 a 100 %. Para el 100 % se escribe *level* = TOP + 1 en lugar de TOP, porque con TOP la señal bajaría durante un paso de cada periodo y nunca llegaría al 100 % real. Al probar el brillo se confirmó lo que se plantea en el marco teórico: el ojo no percibe el brillo de forma lineal, y el cambio de 10 % a 20 % se nota mucho más que el de 80 % a 90 %.

Por esa razón, el comando fade <ms> no recorre el ciclo de trabajo en línea recta, sino que usa una tabla de corrección gamma (γ = 2.2) de 101 valores, calculada como level = round(1000 × (B/100)$^{2.2}$), donde B es el brillo percibido. Por ejemplo, para que el LED se vea "a la mitad" basta con un ciclo de trabajo de 22 %. Un fade completo sube de 0 a 100 % y baja de nuevo en 200 escalones, y cada escalón lo da una alarma de hardware, como un metrónomo que marca el paso sin que el programa tenga que esperar.

Fragmento 2. Callback de la alarma del fade (main.c).

```
static int64_t fade_cb(alarm_id_t id, void *user_data)
{
    if (!g_fade_activo) return 0;

    uint32_t paso = g_fade_paso++;
    // Triangulo: 0..100 (sube) y luego 99..0 (baja)
    uint8_t b = (uint8_t)((paso <= 100u) ? paso : (FADE_PASOS - paso));
g_fade_b = b;
    led_set_level(GAMMA[b]);          // brillo percibido lineal

    if (paso >= FADE_PASOS) {         // termino: LED apagado
g_fade_activo = false;
        g_fade_fin    = true;         // aviso para el bucle principal
return 0;
    }
    return g_fade_paso_us;            // siguiente escalon, sin deriva }
```

Cuando el callback devuelve un valor positivo, el SDK reprograma la alarma respecto al instante en que debía dispararse y no respecto al momento actual, por lo que los pequeños retrasos no se acumulan y la duración total del fade se mantiene.

Tabla 4. *Voltaje promedio teórico y medido en el pin del LED (V = 3.3 V).* *alto*

Duty (%)	level	V teórico (V) prom
25	250	0.825
50	500	1.650
75	750	2.475
100	1000	3.300

2. Buzzer pasivo: tonos y melodías

La función buzzer_set() recibe una frecuencia, calcula el TOP correspondiente y fija *level* = (TOP + 1)/2 para mantener un ciclo de trabajo de 50 %, que es el que produce el tono más limpio. TOP y *level* se actualizan juntos para que, gracias al doble buffer, el cambio de nota ocurra al terminar un periodo completo. Con frecuencia cero, la salida se deja en bajo y el buzzer se calla.

Fragmento 3. Cambio de nota en buzzer_set() (main.c).

```
static void buzzer_set(uint32_t f_cHz)        // frecuencia en centesimas de Hz
{
    if (f_cHz == 0u) {                          // silencio
pwm_set_chan_level(g_slice_buz, g_chan_buz, 0);
g_buz_fcHz = 0;          return;
    }
    uint64_t ticks_x100 = (uint64_t)(g_clk_hz / BUZ_DIV) * 100u;
uint32_t top = (uint32_t)((ticks_x100 + f_cHz / 2u) / f_cHz) - 1u;     if
(top > 65535u) top = 65535u;

    pwm_set_wrap(g_slice_buz, (uint16_t)top);
    pwm_set_chan_level(g_slice_buz, g_chan_buz, (uint16_t)((top + 1u) / 2u));
g_buz_top  = (uint16_t)top;     g_buz_fcHz = f_cHz;
}
```

Las frecuencias se guardan en centésimas de hertz y todos los cálculos se hacen con enteros, de modo que la función puede llamarse desde una interrupción sin depender de la unidad de punto flotante. Sobre esta función se construyó un reproductor de melodías: cada canción es una tabla de pares (frecuencia, duración) y una alarma de hardware cambia de nota cuando termina la anterior. El

reproductor funciona como un tocadiscos al que solo se le cambia el disco: los comandos melodia y tetris usan la misma lógica con tablas distintas. Cuando dos notas iguales van seguidas, se inserta un silencio breve para que se escuchen como dos golpes y no como una sola nota larga.

Tabla 5. *Frecuencia programada y frecuencia medida en el buzzer.*

Comando	f programada (Hz)
tono 262	261.63
tono 440	440.00
tono 1000	1000.00
tono 5000	5000.00

3. Servomotor: posición por ancho de pulso

El servomotor trabaja a 50 Hz y su posición depende solo de la duración del pulso en alto. Siguiendo la convención descrita por Sufyan (2025), un pulso de 1 ms corresponde aproximadamente a 0°, uno de 1.5 ms a 90° y uno de 2 ms a 180°, lo que equivale a ciclos de trabajo de apenas 5 % a 10 %. El comando servo <angulo> convierte el ángulo en microsegundos con una relación lineal y limita el resultado al rango permitido, para evitar que el mecanismo golpee contra sus topes físicos.

Fragmento 4. Conversión de ángulo a ancho de pulso en servo_set_angulo() (main.c).

```
#define SERVO_TOP        19999u   // 20 ms con pasos de 1 us
#define SERVO_MIN_US      1000u   // 0 grados
#define SERVO_MAX_US      2000u   // 180 grados
#define SERVO_CENTRO_US   1500u   // 90 grados

static void servo_set_angulo(uint32_t angulo)
{
    if (angulo > 180u) angulo = 180u;
uint32_t us = SERVO_MIN_US +
                  (angulo * (SERVO_MAX_US - SERVO_MIN_US) + 90u) / 180u;
pwm_set_chan_level(g_slice_srv, g_chan_srv, (uint16_t)us);  // level = us     g_servo_us
= us;
}
```

Como cada paso del contador vale 1 µs, la resolución del servomotor es de 1000 pasos en todo su recorrido, es decir, unos 0.18° por paso, mucho más fino de lo que el propio mecanismo puede reproducir. Los límites de 1000 y 2000 µs son conservadores; algunos SG90 aceptan pulsos más amplios, pero conviene verificarlos con cuidado en cada modelo.

Tabla 6. *Ancho de pulso teórico y medido en el servomotor.*

Ángulo (°)	Pulso teórico (ms)	Duty teórico (%)
0	1.000	5.0
45	1.250	6.25
90	1.500	7.5
135	1.750	8.75

180	2.000	10.0

4. Motor de corriente directa: velocidad por voltaje promedio

El comando motor <duty> fija el ciclo de trabajo del PWM que llega a la compuerta del MOSFET. Cuando el MOSFET conduce, el motor recibe los 5 V de la fuente; cuando se apaga, la corriente sigue circulando por el diodo de rueda libre. Dietrich (2022) explica que, al conmutar a una frecuencia suficientemente alta, la inercia de la carga suaviza los pulsos y el motor responde al voltaje promedio. Por eso, idealmente, $V_{prom} = 5\ V \times D$. En la práctica,
la velocidad también depende de la carga mecánica, de la caída en el MOSFET y de la fricción interna, y existe un ciclo de trabajo mínimo por debajo del cual el motor no logra arrancar.

Fragmento 5. Ajuste de velocidad en motor_set_duty() (main.c).

```
#define MOTOR_TOP  7499u            // 20 kHz con div = 1

static void motor_set_duty(uint8_t duty)
{
    if (duty > 100u) duty = 100u;
    // duty 100 -> level = TOP + 1 -> 100 % real
    uint16_t level = (uint16_t)((uint32_t)duty * (MOTOR_TOP + 1u) / 100u);
pwm_set_chan_level(g_slice_mot, g_chan_mot, level);     g_motor_duty =
duty; }
```

Tabla 7. *Voltaje promedio teórico y medido en el motor (V_{prom} = 5 V).* *fuente*

Duty (%)	V teórico (V) prom
10	0.50
25	1.25
50	2.50
75	3.75
100	5.00

Consola de comandos

La consola habla directamente con el periférico UART0 por medio de uart_getc() y uart_putc_raw(). La función consola_poll() lee todos los caracteres disponibles en la FIFO de recepción y regresa de inmediato si no hay nada. Cada carácter se devuelve como eco, se acumula en un arreglo de 48 caracteres y, al recibir \r o \n, la línea se envía a procesar(), que separa el comando del argumento con sscanf() y valida el rango antes de tocar cualquier slice. Si el valor está fuera de rango, la consola responde con un mensaje de error y la carga conserva su estado anterior.

Tabla 8. *Comandos de la consola serial implementados.*

Comando	Acción	Rango válido
led <duty>	Fija el brillo del LED (detiene el fade)	0 a 100 %
fade <ms>	Sube y baja el brillo con corrección gamma	200 a 60 000 ms
tono <Hz>	Tono fijo en el buzzer; 0 = silencio	229 a 5000 Hz

melodia / tetris	Reproduce una melodía sin bloquear la consola	—
servo <angulo>	Mueve el servomotor al ángulo indicado	0 a 180°
motor <duty>	Fija la velocidad del motor DC	0 a 100 %
stop	Detiene melodía, fade y motor	—
estado	Muestra los valores actuales y las frecuencias reales	—
ayuda	Lista de comandos	—

El comando estado no repite los valores pedidos, sino que calcula la frecuencia real de cada slice a partir del divisor y del TOP cargados, lo cual permitió comparar desde la consola lo programado contra lo que después se midió con el osciloscopio. Cuando el usuario fija un valor manual, como led o tono, primero se detiene el fade o la melodía correspondiente, para que dos funciones no compitan por el mismo pin.

Operación simultánea sin bloqueos

Para que las cuatro cargas y la consola funcionen al mismo tiempo, en ninguna parte del programa se usa sleep_ms(). El trabajo se reparte en tres niveles. El hardware de PWM genera las cuatro señales de forma continua y autónoma. Las alarmas de hardware se encargan de lo que avanza con el tiempo, como el cambio de nota de una melodía o cada escalón del fade, y en sus callbacks solo actualizan registros y banderas, sin imprimir nada. Por último, el bucle principal atiende la consola y muestra los avisos que dejan las alarmas, por ejemplo "melodia terminada".

Fragmento 6. Bucle principal (main.c).

```
while (true) {
    consola_poll();                    // atiende al usuario

    if (g_mel_fin) {                   // avisos que dejan las alarmas
g_mel_fin = false;
        con_printf("\nmelodia terminada\n> ");
    }
    if (g_fade_fin) {
g_fade_fin = false;
        con_printf("\nfade terminado\n> ");
    }
    tight_loop_contents();
}
```

Este esquema se parece a una cocina con varios fogones encendidos: cada fogón (slice) cocina por su cuenta, los temporizadores (alarmas) avisan cuándo hay que voltear algo, y el cocinero (bucle principal) queda libre para atender nuevos pedidos. Como consecuencia, es posible reproducir una melodía mientras corre un fade, mover el servomotor y cambiar la velocidad del motor, todo sin que la música se corte ni la consola deje de responder.

Pruebas en simulación

Antes de armar el circuito, el programa se probó en Wokwi con la Pico 2, el LED, el buzzer y el servomotor. Con el analizador lógico de Wokwi se capturaron las señales de los GPIO 2, 4, 6 y 8 en un archivo VCD, que después se abrió en Surfer para medir el periodo y el ancho de pulso de cada canal

con sus cursores. En esta etapa se verificó que las frecuencias coincidieran con la Tabla 2 y que los comandos fuera de rango se rechazaran.

Pruebas en hardware

Con el circuito físico armado, las mediciones se hicieron con el osciloscopio y el multímetro, tomando la tierra común como referencia. Con el osciloscopio se midieron la frecuencia y el ancho de pulso de cada canal; con el multímetro en modo de voltaje de CD se midió el voltaje promedio en el pin del LED y entre las terminales del motor. El multímetro es útil aquí porque, al responder lentamente, actúa como el ojo o la inercia del motor: promedia la señal en lugar de seguir cada pulso.

Finalmente, se comprobó la operación simultánea de las cuatro cargas. Se inició la melodía con `melodia`, después el fade con `fade 4000` y, mientras ambos seguían en curso, se enviaron `servo 135`, `motor 60` y `estado`. La consola respondió a cada comando sin interrumpir la música ni el fade, y el comando `estado` reportó los valores de todas las cargas al mismo tiempo.

IBERO
Desarrollo PUEBLA

Descripción general del sistema

El sistema está formado por dos programas, cada uno en su propia Raspberry Pi Pico 2. **PICO_PRINCIPAL_EXAMEN** contiene toda la lógica del juego: botones, LEDs, temporizador, medición del tiempo de reacción y consola de comandos por UART0. **PICO_APOYO_EXAMEN** se ejecuta en una segunda tarjeta que no participa en el juego: solo reenvía bytes entre el puerto USB de la computadora y la UART de la Pico principal, como un intérprete que traduce entre dos personas que hablan idiomas distintos.

El juego se opera desde una consola serial. Al encenderse, envía por UART un mensaje de bienvenida con la lista de comandos. Una partida inicia con el comando start: el LED rojo comienza a parpadear con el periodo configurado y, después de un tiempo aleatorio entre 2 y 5 segundos, se apaga y se enciende el LED verde. El primer jugador que presiona su botón con el LED verde encendido gana y el sistema reporta su tiempo de reacción. Si un jugador presiona antes del verde comete una salida en falso y el punto es para su rival; si nadie presiona durante 5 segundos, la partida termina sin ganador.

Tabla 1. *Comandos de la consola UART implementados.*

Comando	Acción	Respuesta
start	Inicia una partida si no hay otra en curso	Listo o Error: partida en curso
set periodo <ms>	Cambia el periodo de parpadeo del LED rojo (50–1000 ms); solo con el juego detenido	Listo o Error: periodo fuera de rango (50-1000 ms)

get stats	Muestra periodo, resultado de la última partida, errores de UART y marcador	Bloque de estadísticas
reset	Reinicia el marcador de ambos jugadores	Listo
Otro texto	—	Error: comando desconocido

Diagrama de conexiones y montaje

La Pico principal se comunica con la computadora a través de la Pico de apoyo, que funciona como puente USB–UART. Las líneas se conectan cruzadas: el GP0 (TX) de la principal va al GP1 (RX) de la de apoyo y el GP0 (TX) de la de apoyo va al GP1 (RX) de la principal, ya que lo que una transmite la otra debe recibirlo; además, ambas tarjetas comparten tierra para tener la misma referencia de voltaje. Cada Pico se alimenta con su propio cable USB, pero solo la de apoyo se usa como puerto serial en la computadora. Los LEDs se conectan en configuración de fuente de corriente (el GPIO entrega corriente al LED cuando está en alto) y los botones se conectan entre el GPIO y tierra, aprovechando la resistencia pull-up interna; por ello, en reposo el pin lee 1 y al presionar se produce un flanco de bajada.

Tabla 2. *Asignación de pines de la Pico principal.*

Pin	Función	Configuración
GP0	UART0 TX	GPIO_FUNC_UART
GP1	UART0 RX	GPIO_FUNC_UART
GP16	LED rojo (preparados)	Salida, activo en alto
GP17	LED verde (ya)	Salida, activo en alto
GP18	Botón jugador 1	Entrada con pull-up, IRQ flanco de bajada
GP19	Botón jugador 2	Entrada con pull-up, IRQ flanco de bajada

Tabla 3. *Conexiones entre la Pico principal y la Pico de apoyo.*

Pico principal	Pico de apoyo	Señal
GP0 (UART0 TX)	GP1 (UART0 RX)	Respuestas y eco hacia la consola
GP1 (UART0 RX)	GP0 (UART0 TX)	Comandos escritos en la consola
GND	GND	Referencia común
—	USB	Puerto serial virtual hacia la computadora

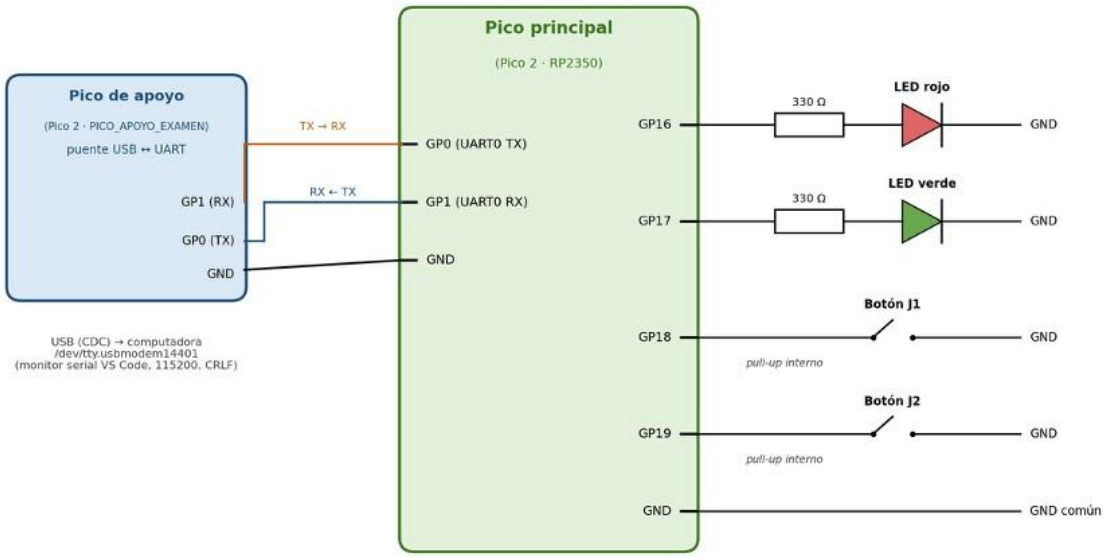


Figura 1. *Diagrama de conexiones de la Pico principal, la Pico de apoyo, los LEDs y los botones.*

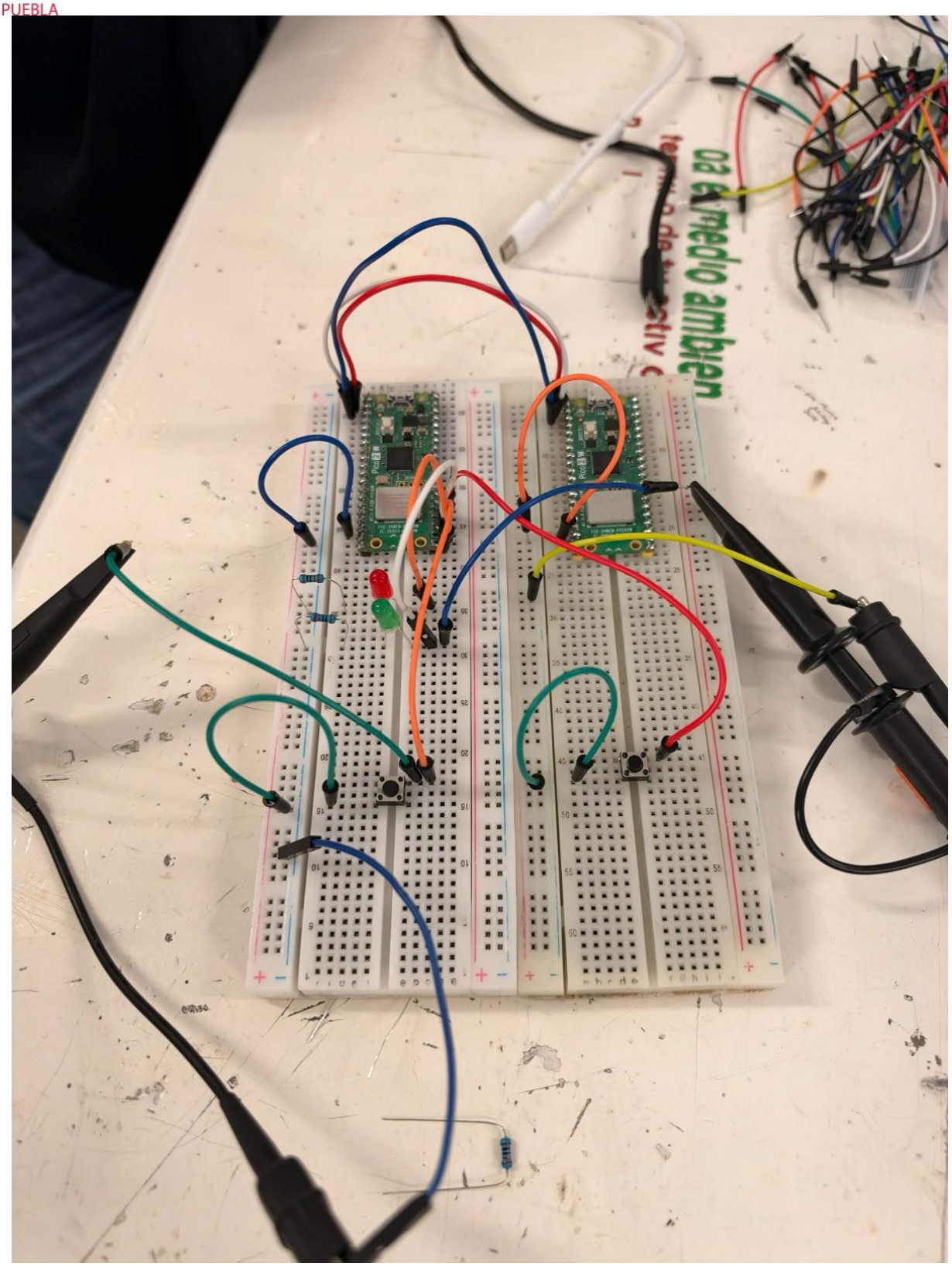


Figura 2. *Fotografía del montaje en protoboard.*

Diagrama de bloques: qué corre en cada ISR y qué corre en el bucle

La arquitectura del programa sigue el modelo *foreground/background*: tres rutinas de interrupción atienden los eventos de hardware en cuanto ocurren y dejan su resultado en

buffers o banderas, mientras que el bucle principal consume esa información sin bloquearse. La Figura 3 muestra esta división y la Tabla 4 la resume.

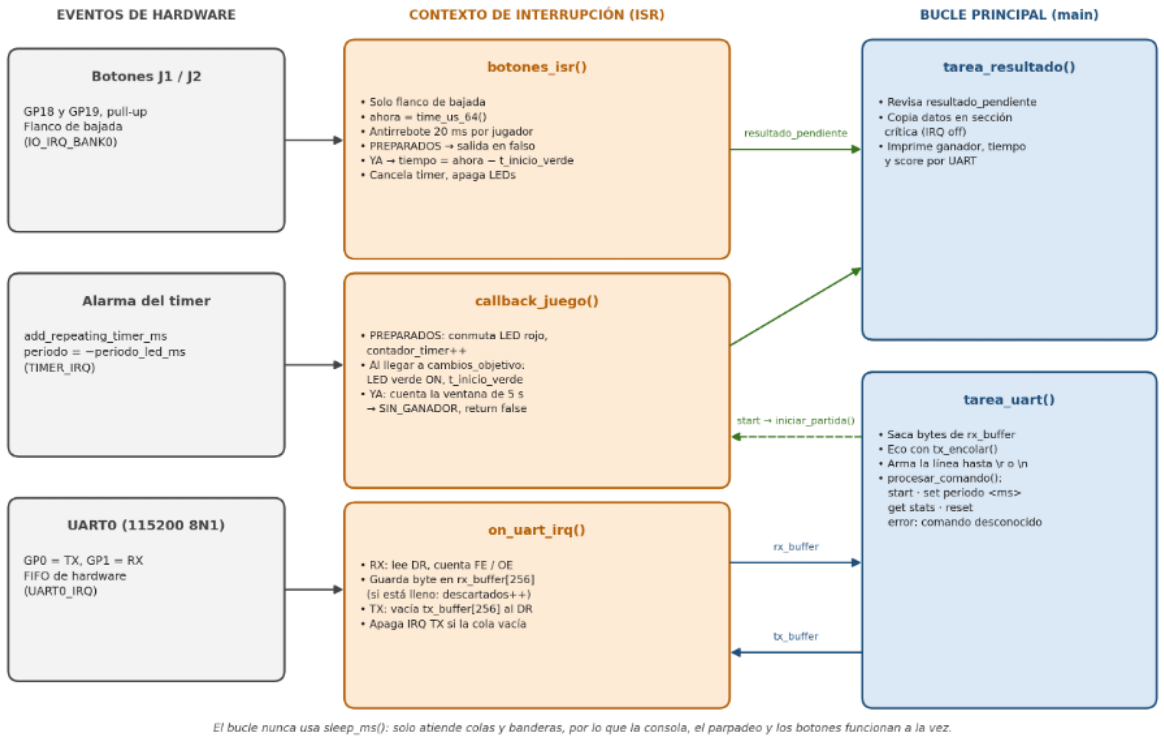


Figura 3. *Diagrama de bloques del sistema con las tareas que se ejecutan en cada ISR y en el bucle principal.*

Tabla 4. *Distribución de responsabilidades entre las ISR y el bucle principal.*

Contexto	Se dispara por	Trabajo que realiza
botones_isr()	Flanco de bajada en GP18 o GP19	Marca de tiempo, antirrebote, decide GANA o FALSO, detiene el timer, actualiza el marcador y levanta resultado_pendiente
callback_juego()	Vencimiento de la alarma del timer repetitivo	Parpadeo del LED rojo, cuenta del tiempo aleatorio, encendido del LED verde con su marca de tiempo y cierre por los 5 s sin respuesta
on_uart_irq()	Byte recibido o espacio libre en la FIFO de TX de UART0	Copia bytes de la FIFO a rx_buffer, cuenta errores y llena la FIFO de TX desde tx_buffer
tarea_uart() (bucle)	Hay bytes en rx_buffer	Eco, armado de la línea e interpretación de comandos
tarea_resultado() (bucle)	resultado_pendiente == true	Formatea e imprime el resultado y el marcador

Ninguna ISR llama a printf() ni a snprintf(): el formateo de texto, que es la operación más lenta, se realiza siempre en el bucle principal. Tampoco se utiliza sleep_ms() en ninguna parte del programa, por lo que los tres flujos (consola, parpadeo y botones) avanzan al mismo tiempo.

El diagrama solo incluye la Pico principal, que es donde se ejecuta el juego. La Pico de apoyo queda fuera del diagrama porque no maneja interrupciones propias: su programa es un único bucle que revisa el USB y la UART por *polling* y reenvía lo que encuentra, como se explica en el Fragmento 13.

Máquina de estados del juego

El estado del juego se guarda en la variable estado_juego, y las transiciones las provocan tres fuentes distintas: la consola (start), el timer (fin del tiempo aleatorio o de la ventana de 5 s) y los botones. La Figura 4 muestra el diagrama de estados.



Figura 4. *Máquina de estados del duelo de reacción con sus tres estados y transiciones.*

Fragmentos de código explicados

1. Configuración de botones, UART e interrupciones

Fragmento 1. Inicialización en main() (líneas 687–717 de PICO_PRINCIPAL_EXAMEN.c)

```c
int main(void) {

    // LEDs
    gpio_init(LED_ROJO);
    gpio_set_dir(LED_ROJO,GPIO_OUT);
    gpio_put(LED_ROJO,0);
    gpio_init(LED_VERDE);
    gpio_set_dir(LED_VERDE,GPIO_OUT);
    gpio_put(LED_VERDE,0);

    // Botones
    gpio_init(BOTON_J1);
    gpio_set_dir(BOTON_J1,GPIO_IN);
    gpio_pull_up(BOTON_J1);
    gpio_init(BOTON_J2);
    gpio_set_dir(BOTON_J2,GPIO_IN);
    gpio_pull_up(BOTON_J2);

    // Interrupcion por botones con callback
    gpio_set_irq_enabled_with_callback(
        BOTON_J1,
        GPIO_IRQ_EDGE_FALL,
        true,
        &botones_isr
    );

    // Habilitar interrupción por edge fall
    gpio_set_irq_enabled(BOTON_J2,GPIO_IRQ_EDGE_FALL,true);

    // Set UART
    gpio_set_function(TX_PIN,GPIO_FUNC_UART);
    gpio_set_function(RX_PIN,GPIO_FUNC_UART);

    // Inicialización UART
    uart_init(UART_ID,BAUD_RATE);

    // Configuración 8N1
    uart_set_format(UART_ID,8,1,UART_PARITY_NONE);

    // FIFO UART habilitada
    uart_set_fifo_enabled(UART_ID,true);

    // Interrupciones UART
    irq_set_exclusive_handler(UART0_IRQ,on_uart_irq);
    irq_set_enabled(UART0_IRQ,true);
    uart_set_irq_enables(UART_ID,true,false);
```

Los botones se configuran como entradas con pull-up interno y con interrupción por flanco de bajada. gpio_set_irq_enabled_with_callback() registra botones_isr como el *callback* único del banco de GPIO; para el segundo botón basta con gpio_set_irq_enabled(), ya que comparte el mismo *callback*. Después se asignan GP0 y GP1 a la función UART, se inicializa UART0 a 115200 baudios con formato 8N1 y se habilita su FIFO de hardware. Por último se registra on_uart_irq como manejador exclusivo de UART0_IRQ, se habilita la interrupción en el NVIC y, en el periférico, solo la interrupción de recepción (uart_set_irq_enables(UART_ID, true, false)); la de transmisión se enciende únicamente cuando hay datos que enviar.

2. ISR de UART y buffer circular de recepción

Fragmento 2. Rutina de interrupción de UART0 (líneas 159–205 de PICO_PRINCIPAL_EXAMEN.c)

```c
static void on_uart_irq(void) {

    // Recepción
    while (uart_is_readable(UART_ID)) {

        // Leer registro de datos UART
        uint32_t dr = uart_get_hw(UART_ID)->dr;

        // Revisar framing error
        if (dr & UART_UARTDR_FE_BITS) {
            framing_errors++;
        }

        // Revisar overrun
        if (dr & UART_UARTDR_OE_BITS) {
            overruns++;
        }

        // Obtener únicamente los 8 bits del dato
        uint8_t c = (uint8_t)(dr & 0xFFu);

        // Calcular siguiente posición
        uint16_t siguiente = (rx_head + 1) % RX_BUF_SIZE;

        // Si existe espacio
        if (siguiente != rx_tail) {
            rx_buffer[rx_head] = c;
            rx_head = siguiente;
        }

        // Buffer circular lleno
        else {
            descartados++;
        }
    }

    // Transmisión
    while (uart_is_writable(UART_ID) && tx_tail != tx_head) {
        uart_get_hw(UART_ID)->dr = tx_buffer[tx_tail];
        tx_tail = (tx_tail + 1) % TX_BUF_SIZE;
    }

    // Si ya no queda nada por transmitir
    if (tx_tail == tx_head) {
        uart_set_irq_enables(UART_ID,true,false);
    }
}
```

En la parte de recepción, la ISR vacía la FIFO de hardware mientras uart_is_readable() sea verdadero. Cada lectura del registro DR entrega el byte en los 8 bits menos significativos y, en los bits superiores, las banderas de error; por eso se revisan UART_UARTDR_FE_BITS y UART_UARTDR_OE_BITS para contar *framing errors* y *overruns* antes de quedarse con dr & 0xFF. El byte se guarda en rx_buffer solo si hay espacio; si el buffer de software está lleno, se incrementa descartadosen lugar de sobrescribir datos. En la parte de transmisión, la ISR copia bytes de tx_buffer al registro de datos mientras la FIFO de TX tenga espacio y, cuando la cola queda vacía, deshabilita la interrupción de TX para no entrar en un ciclo infinito de interrupciones.

3. Transmisión no bloqueante

Fragmento 3. Función tx_encolar() (líneas 113–148 de PICO_PRINCIPAL_EXAMEN.c)

```c
static void tx_encolar(uint8_t c) {

    // Evitar que la ISR UART modifique tx_tail mientras estamos actualizando el buffer
    uint32_t estado_irq = save_and_disable_interrupts();

    // Saber si la cola estaba vacía antes de agregar el dato
    bool cola_vacia = (tx_head == tx_tail);

    // Calcular siguiente posición del buffer circular
    uint16_t siguiente = (tx_head + 1) % TX_BUF_SIZE;

    // Si siguiente != tail, existe espacio
    if (siguiente != tx_tail) {

        // Guardar carácter
        tx_buffer[tx_head] = c;

        // Avanzar head
        tx_head = siguiente;

        // Si la cola estaba completamente vacía,
        // mandamos directamente el primer byte al UART.
        if (cola_vacia && uart_is_writable(UART_ID)) {
            uart_get_hw(UART_ID)->dr = tx_buffer[tx_tail];
            tx_tail = (tx_tail + 1) % TX_BUF_SIZE;
        }

        // Si quedan datos habilitar la IRQ de TX
        if (tx_tail != tx_head) {
            uart_set_irq_enables(UART_ID, true, true);
        }
        else {
            uart_set_irq_enables(UART_ID, true, false);
        }
    }

    // Restaurar interrupciones
    restore_interrupts(estado_irq);
}
```

Toda la salida por UART pasa por esta función, que coloca el carácter en tx_buffer en lugar de esperar a que la UART termine de enviarlo; así, imprimir un mensaje largo no detiene el programa. Como tanto el bucle principal como la ISR modifican tx_tail, la actualización se protege con save_and_disable_interrupts(). Si la cola estaba vacía se escribe el primer byte directamente en el registro de datos para "arrancar" la transmisión, y después se habilita la interrupción de TX solo si quedan bytes pendientes. Si el buffer está lleno, el carácter se descarta en lugar de bloquear.

4. Consola: eco, armado de línea y comandos

Fragmento 4. Tarea de consola en el bucle principal (líneas 646–675 de PICO_PRINCIPAL_EXAMEN.c)

```c
static void tarea_uart(void) {

    // Vaciar completamente el buffer RX
    while (rx_tail != rx_head) {
        uint8_t c = rx_buffer[rx_tail];
        rx_tail = (rx_tail + 1) % RX_BUF_SIZE;

        // Eco
        tx_encolar(c);

        // Fin de línea
        if (c == '\r' || c == '\n') {
            linea[n] = '\0';

            // Evitar procesar línea vacía
            if (n > 0) {
                procesar_comando(linea);
            }

            n = 0;
        }

        // Seguir acumulando caracteres
        else {
            if (n < LINE_SIZE - 1) {
                linea[n] = (char)c;
                n++;
            }
        }
    }
}
```

tarea_uart() consume todos los bytes disponibles en rx_buffer. Cada carácter se reenvía como eco para que el usuario vea lo que escribe y se acumula en linea, con un límite de 63 caracteres para no desbordar el arreglo. Al recibir \r o \n se cierra la cadena con \0 y se envía a procesar_comando(); las líneas vacías se ignoran, lo que evita procesar dos veces un fin de línea CRLF.

Fragmento 5. Comando set periodo (líneas 608–640 de PICO_PRINCIPAL_EXAMEN.c)

```
if (strncmp(cmd, "set periodo", 11) == 0) {

    // Candado: partida en curso
    if (estado_juego != DETENIDO) {
        tx_encolar_texto(" Error: partida en curso\r\n");
        return;
    }

    // Asignar valor del periodo
    unsigned int nuevo_periodo;
    char extra;

    int encontrados =
        sscanf(
            cmd,
            "set periodo %u %c",
            &nuevo_periodo,
            &extra
        );

    if (encontrados == 1) {

        // Comprobar rango 50-1000 ms
        if (nuevo_periodo >= 50 && nuevo_periodo <= 1000) {
            periodo_led_ms = nuevo_periodo;
            tx_encolar_texto(" Listo\r\n");
        }
        else {
            tx_encolar_texto(
                " Error: periodo fuera de rango (50-1000 ms)\r\n"
            );
        }
    }
    else {
        tx_encolar_texto(" Error: comando desconocido\r\n");
    }

    return;
}
```

El comando se reconoce con strncmp() y el valor se extrae con sscanf(). El especificador adicional %c detecta texto sobrante después del número: solo si sscanf() devuelve exactamente 1 el comando está bien formado. El periodo se valida en el rango de 50 a 1000 ms y no puede cambiarse con una partida en curso, ya que el timer ya se configuró con el valor anterior.

5. Inicio de partida y tiempo aleatorio

Fragmento 6. Cálculo del tiempo aleatorio en iniciar_partida() (líneas 294–314 de PICO_PRINCIPAL_EXAMEN.c)

```
periodo_timer_actual_ms = periodo_led_ms;

// Calcular el mínimo de callbacks equivalente a 2 s.
uint32_t min_cambios =
    (ESPERA_MIN_MS + periodo_timer_actual_ms - 1u)
    / periodo_timer_actual_ms;

// Calcular el máximo de callbacks equivalente a 5 s.
uint32_t max_cambios =
    ESPERA_MAX_MS / periodo_timer_actual_ms;

// Seguridad
if (max_cambios < min_cambios) {
    max_cambios = min_cambios;
}

// Tiempo aleatorio
cambios_objetivo =
    min_cambios +
    (rand() % (max_cambios - min_cambios + 1u));

contador_timer = 0;
estado_juego = PREPARADOS;

// Timer repetitivo
timer_activo = add_repeating_timer_ms(
    -(int32_t)periodo_timer_actual_ms,
    callback_juego,
    NULL,
    &timer_juego
);
```

El tiempo de espera antes del LED verde se expresa como un número entero de ejecuciones del *callback* del timer. Se calcula el mínimo de ejecuciones que cubre 2 s (redondeando hacia arriba) y el máximo que cabe en 5 s (redondeando hacia abajo), y se elige un valor aleatorio entre ambos con rand(). Con el periodo de 350 ms usado en las pruebas, el LED verde aparece después de 6 a 14 ejecuciones, es decir, entre 2.1 y 4.9 s. Finalmente se arma el timer con periodo negativo para que el intervalo entre ejecuciones no se alargue con la duración del *callback*.

6. Callback del timer

Fragmento 7. Estado PREPARADOS en callback_juego() (líneas 213–242 de PICO_PRINCIPAL_EXAMEN.c)

```
if (estado_juego == PREPARADOS) {

    // Invertir estado del LED rojo
    rojo_encendido = !rojo_encendido;
    gpio_put(LED_ROJO, rojo_encendido);

    // Contar una ejecución del timer
    contador_timer++;

    // Si se cumplió el tiempo aleatorio...
    if (contador_timer >= cambios_objetivo) {

        // Configuración LEDs
        rojo_encendido = false;
        gpio_put(LED_ROJO, 0);
        gpio_put(LED_VERDE, 1);

        // Tomar timestamp del inicio del verde
        t_inicio_verde = time_us_64();

        // Cambiar estado del juego
        estado_juego = YA;

        // Reiniciar contador para los 5 s
        contador_timer = 0;
    }

    // true significa que el timer continúa
    return true;
}
```

En el estado PREPARADOS, cada ejecución invierte el LED rojo y aumenta contador_timer. Cuando el contador alcanza cambios_objetivo, se apaga el rojo, se enciende el verde y en la línea siguiente se toma t_inicio_verde con time_us_64(); esta marca es el punto de partida de la medición del tiempo de reacción. El estado cambia a YA y el contador se reinicia para medir la ventana de respuesta.

Fragmento 8. Estado YA en callback_juego() (líneas 245–272 de PICO_PRINCIPAL_EXAMEN.c)

```
if (estado_juego == YA) {
    contador_timer++;

    // Calcular callbacks para representar 5s.
    uint32_t ciclos_5s =
        (VENTANA_VERDE_MS + periodo_timer_actual_ms - 1u)
        / periodo_timer_actual_ms;

    // Verde + de 5s.
    if (contador_timer >= ciclos_5s) {

        // Apagar LED verde
        gpio_put(LED_VERDE, 0);

        // Guardar resultado
        resultado = SIN_GANADOR;
        ganador = 0;
        jugador_falso = 0;
        tiempo_reaccion_us = 0;

        // Regresar al estado inicial
        estado_juego = DETENIDO;
        timer_activo = false;

        resultado_pendiente = true;

        // false detiene el repeating timer
        return false;
    }

    return true;
}
```

En el estado YA el mismo timer cuenta cuántas ejecuciones equivalen a 5 s. Si se alcanza ese número sin que nadie presione, se apaga el LED verde, se registra SIN_GANADOR y el *callback* devuelve false, lo que detiene el timer repetitivo. Dado que la ventana se mide en múltiplos del periodo, su duración real es de ⌈5000 / periodo⌉ × periodo; con 350 ms resultan 15 ejecuciones, es decir, 5.25 s.

7. ISR de los botones: antirrebote y medición

Fragmento 9. Identificación del botón y antirrebote (líneas 329–351 de PICO_PRINCIPAL_EXAMEN.c)

```
static void botones_isr(uint gpio, uint32_t events) {

    if (!(events & GPIO_IRQ_EDGE_FALL)) {
        return;
    }

    uint64_t ahora = time_us_64();

    uint8_t jugador;

    if (gpio == BOTON_J1) {
        jugador = 1;

        if (ultimo_j1_us != 0 &&
            ahora - ultimo_j1_us < DEBOUNCE_US) {
            return;
        }

        ultimo_j1_us = ahora;
    }

    else if (gpio == BOTON_J2) {
        jugador = 2;

        if (ultimo_j2_us != 0 &&
            ahora - ultimo_j2_us < DEBOUNCE_US) {
            return;
        }

        ultimo_j2_us = ahora;
    }

    else {
        return;
    }
}
```

Lo primero que hace la ISR es tomar la marca de tiempo ahora, para que el cálculo posterior no incluya el tiempo de las demás instrucciones. Después identifica al jugador por el número de GPIO y aplica el antirrebote: si han pasado menos de 20 ms desde la última pulsación aceptada de ese mismo botón, la interrupción se ignora. Cada jugador tiene su propio registro de tiempo, así que el rebote de un botón no bloquea al otro.

Fragmento 10. Respuesta válida en el estado YA (líneas 393–425 de PICO_PRINCIPAL_EXAMEN.c)

```
if (estado_juego == YA) {

    // Detener timer
    if (timer_activo) {
        cancel_repeating_timer(&timer_juego);
        timer_activo = false;
    }

    // Apagar LEDs
    gpio_put(LED_VERDE, 0);
    gpio_put(LED_ROJO, 0);
    rojo_encendido = false;

    // Guardar ganador
    ganador = jugador;
    jugador_falso = 0;

    // Medición del tiempo de reacción
    tiempo_reaccion_us = ahora - t_inicio_verde;
    resultado = GANA;

    // Actualizar score
    if (jugador == 1) {
        score_j1++;
    }
    else {
        score_j2++;
    }

    // Terminar partida
    estado_juego = DETENIDO;
    resultado_pendiente = true;
}
```

Si la pulsación llega en el estado YA, se cancela el timer para que no declare SIN_GANADOR, se apagan los LEDs, se registra al ganador y se calcula tiempo_reaccion_us = ahora − t_inicio_verde. Como el estado pasa inmediatamente a DETENIDO, la pulsación del segundo jugador que llegue después se ignora. El caso PREPARADOS (líneas 359–390) es análogo, pero asigna el punto al rival y deja el tiempo en cero, ya que no existe una reacción válida. En ambos casos la ISR solo levanta la bandera resultado_pendiente; no imprime nada.

8. Reporte del resultado y bucle principal

Fragmento 11. Copia protegida de los datos del resultado (líneas 511–528 de PICO_PRINCIPAL_EXAMEN.c)

```
// Mostrar automáticamente el resultado al terminar una partida
static void tarea_resultado(void) {

    // Si no hay resultado nuevo, no hacer nada
    if (!resultado_pendiente) {
        return;
    }

    // Copiar datos compartidos
    uint32_t estado_irq = save_and_disable_interrupts();
    estado_resultado_t r = resultado;
    uint8_t g = ganador;
    uint8_t jf = jugador_falso;
    uint64_t tiempo = tiempo_reaccion_us;
    uint32_t s1 = score_j1;
    uint32_t s2 = score_j2;

    resultado_pendiente = false;
    restore_interrupts(estado_irq);
```

tarea_resultado() revisa la bandera y, si hay un resultado nuevo, copia en variables locales todos los datos compartidos dentro de una sección crítica. Así se garantiza que el ganador, el tiempo de 64 bits y el marcador que se imprimen pertenecen a la misma partida, aunque una interrupción ocurra durante el formateo. El texto se arma con snprintf() fuera de la sección crítica, para mantener las interrupciones deshabilitadas el menor tiempo posible.

Fragmento 12. Bucle principal (líneas 739–742 de PICO_PRINCIPAL_EXAMEN.c)

```
while (true) {
    tarea_uart();
    tarea_resultado();
}
```

El bucle principal solo llama a dos tareas cortas que nunca esperan: si no hay bytes ni resultados pendientes, regresan de inmediato. Esta estructura es la que permite que las tres actividades del sistema ocurran a la vez.

9. Programa de la Pico de apoyo: puente USB–UART

Fragmento 13. Configuración de la Pico de apoyo (líneas 13–30 de PICO_APOYO_EXAMEN.c)

```
int main(void) {

    // Inicializa comunicación por usb
    stdio_init_all();

    // Configuración de los pines UART
    gpio_set_function(TX_PIN,GPIO_FUNC_UART);
    gpio_set_function(RX_PIN,GPIO_FUNC_UART);

    // Inicializa los pines como tipo UART
    uart_init(UART_ID,BAUD_RATE);

    // configurar formato UART
    uart_set_format(UART_ID,8,1,UART_PARITY_NONE);

    // Habilitar FIFO de la UART
    uart_set_fifo_enabled(UART_ID,true);
```

La Pico de apoyo inicializa stdio con stdio_init_all(); como en su CMakeLists.txt está activado pico_enable_stdio_usb y desactivado pico_enable_stdio_uart, la entrada y salida estándar viajan por USB y la tarjeta aparece en la computadora como /dev/tty.usbmodem14401, el puerto que se observa en las capturas del monitor serial. Después configura su UART0 en GP0 y GP1 con exactamente los mismos parámetros que la Pico principal (115200 baudios, 8N1 y FIFO habilitada); si alguno de estos valores fuera distinto, los bytes llegarían corruptos y se contarían como *framing errors*.

Fragmento 14. Bucle de reenvío en ambos sentidos (líneas 32–48 de PICO_APOYO_EXAMEN.c)

```
while (true) {

    // CONSOLA - UART APOYO - UART PRINCIPAL

    int ch = getchar_timeout_us(0);
    if (ch != PICO_ERROR_TIMEOUT) {
        uart_putc_raw(UART_ID,(char)ch);    // Llego caracter y lo manda a la pico principal
    }

    // UART PRINCIPAL - UART APOYO - CONSOLA
    while (uart_is_readable(UART_ID)) {

        // Leer un byte recibido desde la principal
        char c = uart_getc(UART_ID);
        putchar(c);     // Mandarlo a consola
    }
}
```

El bucle realiza dos tareas en cada vuelta. En el sentido computadora → Pico principal, getchar_timeout_us(0) revisa si llegó un carácter por USB sin esperar (el tiempo límite es cero); si lo hay, lo envía tal cual por la UART con uart_putc_raw(), que no hace conversiones de fin de línea, de modo que el \r\n del monitor llega intacto. En el sentido Pico principal → computadora, mientras uart_is_readable() indique que hay bytes en la FIFO de recepción, se leen con uart_getc()y se reenvían al USB con putchar(). Como ninguna de las dos operaciones bloquea, el puente atiende ambas direcciones de forma alternada y muy rápida;

a 115200 baudios llega un byte cada 87 µs aproximadamente, mucho más tiempo del que tarda una vuelta del bucle, por lo que la FIFO de 32 bytes nunca se llena. El resultado es transparente: para la Pico principal es como si estuviera conectada directamente a la computadora.

Pruebas de funcionamiento por consola UART

Las pruebas se realizaron con el monitor serial de Visual Studio Code a 115200 baudios y terminación CRLF, conectado al puerto USB de la Pico de apoyo (/dev/tty.usbmodem14401); todo lo que aparece en las capturas pasó por el puente descrito en los Fragmentos 13 y 14. La Figura 5 muestra el mensaje de bienvenida que el sistema envía al arrancar.

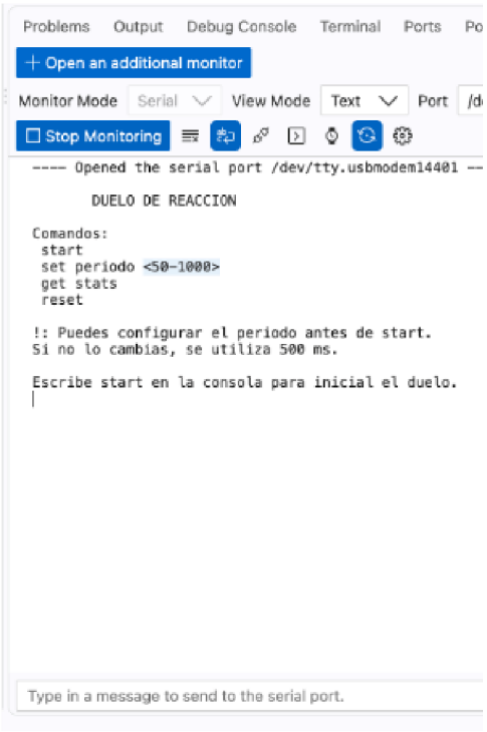


Figura 5. *Mensaje de inicio del duelo de reacción en el monitor serial.*

En la Figura 6 se prueba la validación de comandos: set perido 30 (con error de escritura) se rechaza como comando desconocido, set periodo 30 se rechaza por estar fuera de rango y set periodo 350 se acepta. El comando get stats confirma el nuevo periodo, que aún no hay partidas y que no se han registrado errores de UART.



Figura 6. *Validación de comandos y consulta de estadísticas antes de la primera partida.*

La Figura 7 muestra una partida completa con respuesta válida: el jugador 2 presionó primero con un tiempo de reacción de 558.393 ms, el marcador se actualizó a J1 0, J2 1 y get stats reporta el mismo resultado.

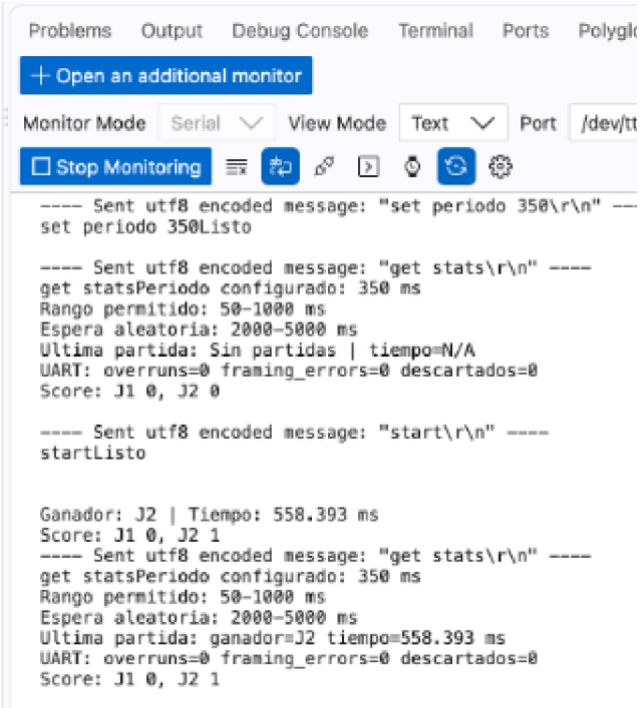


Figura 7. *Partida ganada por el jugador 2 con un tiempo de reacción de 558.393 ms.*

En la Figura 8 se prueban los otros dos desenlaces: en la segunda partida el jugador 2 presionó durante el parpadeo rojo, por lo que se registró una salida en falso y el punto fue para el jugador 1; en la tercera nadie presionó y, al terminar la ventana, el sistema reportó "Sin ganador" sin modificar el marcador.

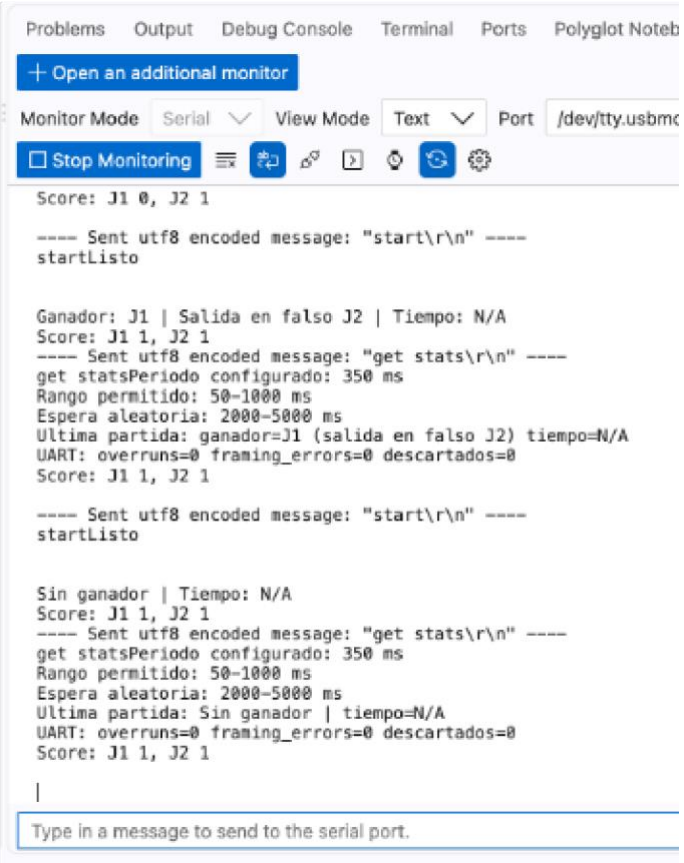


Figura 8. *Salida en falso del jugador 2 y partida sin ganador.*

Finalmente, la Figura 9 muestra que reset reinicia el marcador a cero sin borrar el periodo configurado ni el resultado de la última partida.



Figura 9. *Reinicio del marcador con el comando reset.*

Tabla 5. *Resumen de las pruebas realizadas por consola.*

Prueba	Resultado esperado	Resultado obtenido

Comando mal escrito	Error: comando desconocido	Correcto
set periodo 30	Error por rango	Correcto
set periodo 350	Listo y periodo actualizado	Correcto
Respuesta con LED verde	Ganador y tiempo en ms	J2, 558.393 ms
Pulsación durante el rojo	Salida en falso, punto al rival	Falso J2, gana J1
Sin pulsación	Sin ganador tras la ventana	Correcto
reset	Marcador en 0	Correcto
Errores de UART	0 overruns, 0 framing, 0 descartados	0 / 0 / 0 en todas las consultas

En las capturas se observa que la respuesta de cada comando aparece en la misma línea que el eco (por ejemplo, startListo). Esto ocurre porque el programa procesa el comando al recibir \r y encola la respuesta antes del eco de \n, y el monitor no interpreta un \r aislado como salto de línea. No afecta al funcionamiento, pero podría corregirse enviando \r\nantes de cada respuesta.

Verificación del tiempo de reacción con osciloscopio

Para validar la medición se utilizó un osciloscopio Tektronix MDO3022. El canal 1 (amarillo) se conectó al pin GP17, que enciende el LED verde, y el canal 2 (azul) al pin del botón del jugador, con la referencia de ambas puntas en la tierra común. El disparo se configuró en el flanco de subida del canal 1, es decir, en el instante en que se enciende el LED verde. La base de tiempo fue de 400 ms/div con 10 000 puntos a 2.5 kS/s, lo que da una ventana de 4 s para capturar la partida completa y una resolución de 0.4 ms entre muestras.

Con esta conexión, el tiempo de reacción real es el intervalo entre el flanco de subida del canal 1 (el LED verde se enciende) y el flanco de bajada del canal 2 (el botón se presiona y, por el pull-up interno, el pin pasa de 3.3 V a 0 V). Es como cronometrar una carrera con dos fotos: una cuando se da la señal de salida y otra cuando el corredor cruza la meta. En las capturas, el canal 1 muestra un nivel de aproximadamente 0.31 V en lugar de 3.3 V; esto se debe a que la atenuación configurada en el canal no coincidía con la de la punta (×10), por lo que la amplitud aparece dividida entre diez. Este factor solo afecta la escala vertical y no modifica las mediciones de tiempo.

Primera medición: modo Roll

La primera partida se capturó en modo *Roll*, con el canal 1 a 20 mV/div (Figura 10). Con esa escala, cuando el LED verde se enciende la señal sale de la pantalla por la parte superior y solo vuelve a aparecer cuando el LED se apaga; justo en ese momento el canal 2 muestra el pulso bajo producido por el botón. Los cursores colocados en ambos extremos marcan **Δt = 216.0 ms**. Como los flancos del canal 1 quedan fuera de la pantalla, la posición de los cursores es menos precisa, por lo que se repitió la medición con una escala más adecuada.



Figura 10. *Primera medición en modo Roll con un tiempo entre el encendido del LED verde y la pulsación de 216.0 ms.*

Segunda medición: tiempo de reacción

En la segunda partida, el canal 1 se ajustó a 100 mV/div y la adquisición se detuvo después del evento. La Figura 11 muestra la vista completa: el canal 1 permanece en alto mientras el LED verde está encendido y regresa a cero exactamente cuando el canal 2 presenta el pulso bajo del botón. Esto confirma el comportamiento del Fragmento 10: la ISR del botón apaga el LED verde en cuanto detecta la pulsación válida.

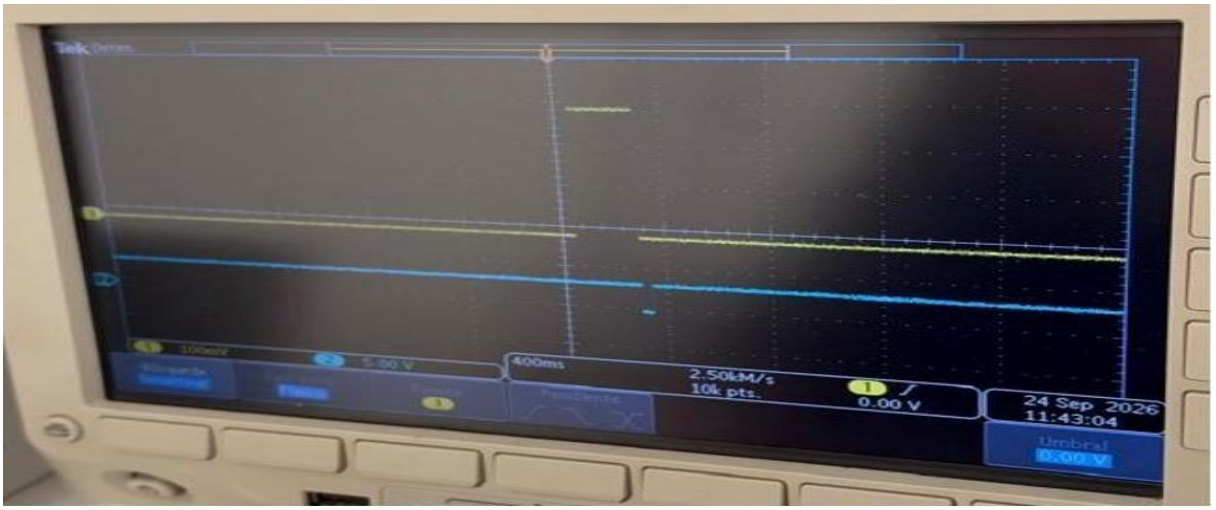


Figura 11. *Vista completa de la segunda partida con el LED verde en el canal 1 y el botón en el canal 2.*

Para medir con mayor precisión se utilizó la ampliación ×4 (100 ms/div) centrada en el evento (Figura 12). El cursor **a** se colocó en el flanco de subida del LED verde (58.40 ms) y el cursor **b** en el flanco de bajada, que coincide con el inicio de la pulsación (302.4 ms). La diferencia entre ambos es **Δt = 244.0 ms**, que corresponde al tiempo de reacción del jugador en esta partida.



Figura 12. *Medición del tiempo de reacción con cursores entre el encendido del LED verde y la pulsación, con Δt = 244.0 ms.*

Duración de la pulsación

Sobre la misma adquisición, los cursores se movieron al canal 2 (Figura 13). El cursor **a** marca el flanco de bajada del botón (302.4 ms) y el cursor **b** su flanco de subida al soltarlo (345.4 ms), por lo que el botón permaneció presionado **43.0 ms**. Solo el flanco de bajada genera una interrupción (GPIO_IRQ_EDGE_FALL), así que la liberación del botón no provoca ningún evento en el programa. Además, la pulsación dura más que la ventana de antirrebote de 20 ms, por lo que cualquier rebote al presionar queda cubierto por ella. Con una resolución de 0.4 ms no es posible observar el rebote del contacto, que dura típicamente menos de un milisegundo; para verlo se necesitaría una base de tiempo del orden de microsegundos.

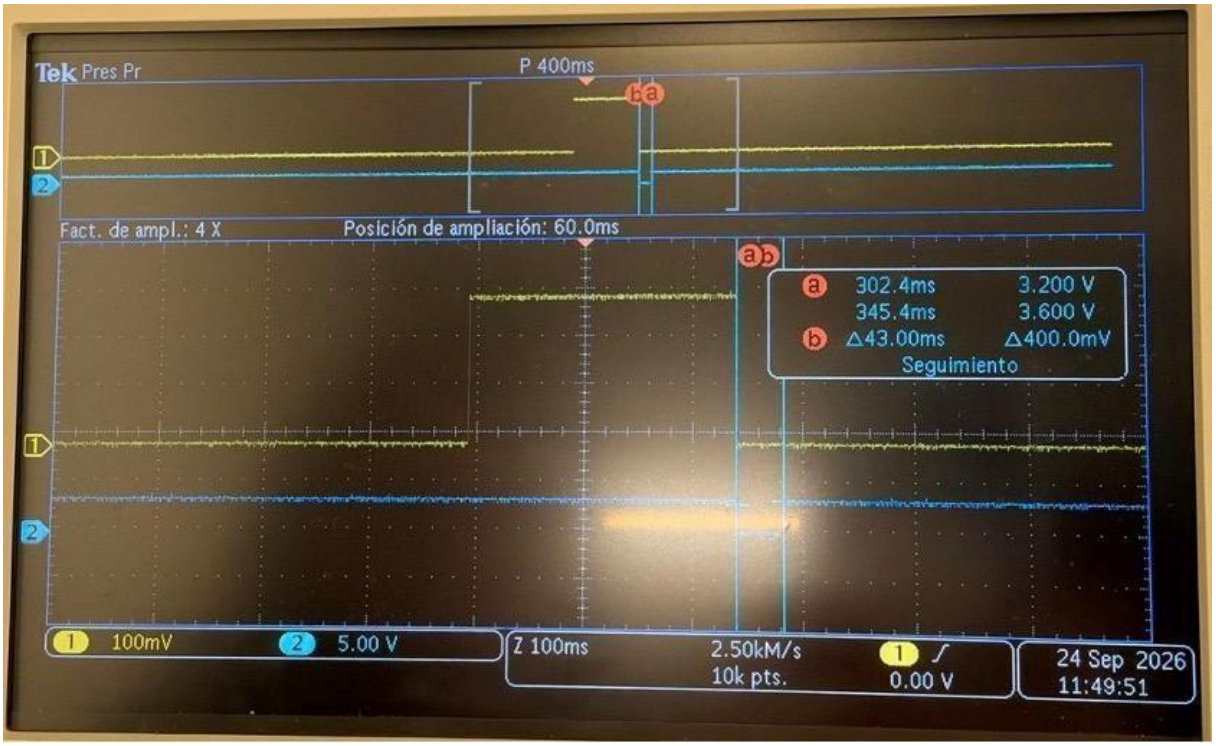


Figura 13. *Medición de la duración de la pulsación en el canal 2 con Δt = 43.0 ms.*

Comparación con la consola UART

Tabla 6. *Comparación entre el tiempo medido por el osciloscopio.*

Partida	Captura	Osciloscopio (ms)
1	Figura 10	216.0
2	Figuras 11 y 12	244.0

Ambos tiempos de reacción, 216.0 ms y 244.0 ms, se encuentran dentro del rango típico de la reacción humana ante un estímulo visual, de 200 a 300 ms aproximadamente. La precisión de la comparación está limitada por el osciloscopio y no por la Pico: con 2.5 kS/s cada muestra representa 0.4 ms y los cursores se colocan manualmente, por lo que la medición tiene una incertidumbre de alrededor de ±1 ms. En cambio, las fuentes de error del programa (la instrucción entre `gpio_put(LED_VERDE, 1)` y la lectura de `t_inicio_verde`, la latencia de entrada a la ISR y la resolución de 1 µs de `time_us_64()`) son de microsegundos. Por lo tanto, se espera que el valor reportado por la consola coincida con el del osciloscopio con una diferencia menor a 1 ms.

Conclusiones

Se implementó un duelo de reacción para dos jugadores en la Raspberry Pi Pico 2 que integra GPIO, interrupciones, un temporizador de hardware y una consola UART, cumpliendo con el requisito de que todas las funciones operen simultáneamente. La clave fue dividir el trabajo: las ISR solo registran eventos, toman marcas de tiempo y actualizan el estado, mientras que el bucle principal se encarga del texto y los comandos. Gracias a ello, el programa no utiliza ninguna espera bloqueante y la consola permanece disponible incluso durante una partida. La Pico de apoyo, por su parte, permitió usar la UART por hardware de

la Pico principal desde la computadora con un programa mínimo de reenvío, lo que separó la comunicación con el usuario de la lógica del juego.

La máquina de estados permitió que una misma pulsación se interpretara de forma distinta según el momento del juego, y reutilizar un solo timer repetitivo para el parpadeo, el tiempo aleatorio y la ventana de 5 s simplificó el diseño. Por su parte, los buffers circulares de recepción y transmisión desacoplaron la velocidad de la UART del resto del programa: en todas las pruebas se registraron cero *overruns*, cero errores de trama y cero bytes descartados.

Las pruebas por consola verificaron los tres desenlaces posibles (victoria con tiempo de reacción, salida en falso y partida sin ganador), la validación de comandos y el reinicio del marcador. Como mejoras se proponen: enviar un salto de línea antes de cada respuesta para separar el eco del texto, medir la ventana de 5 s con una marca de tiempo en lugar de un número de ejecuciones del timer para que no dependa del periodo configurado, y generar la semilla aleatoria a partir de un evento del usuario (por ejemplo, el instante en que se recibe start), ya que al arrancar el valor de time_us_64() es casi siempre el mismo.

Herramienta	Uso específico
ChatGPT	Apoyo en la escritura del código
Gemini	Apoyo en el codigo
Claude	Apoyo en la redacción y organización del documento del examen

Referencias

ARM Limited. (2007). PrimeCell UART (PL011) technical reference manual (Revisión r1p5, ARM DDI 0183G). https://developer.arm.com/documentation/ddi0183/g freeCodeCamp. (2020, 6 de enero). Finite state machine explained. freeCodeCamp.org. https://www.freecodecamp.org/news/finite-state-machines/

Ganssle, J. G. (2014). A guide to debouncing, or, how to debounce a contact in two easy pages (Rev. 4). The Ganssle Group. https://www.ganssle.com/debouncing.htm

h.larsen. (2023, 15 de mayo). Use Pico's USB port as a UART for host communication [Publicación en foro]. Raspberry Pi Forums. https://forums.raspberrypi.com/viewtopic.php?t=351506

Jain, A., Bansal, R., Kumar, A., & Singh, K. D. (2015). A comparative study of visual and auditory reaction times on the basis of gender and physical activity levels of medical first year students. International Journal of Applied and Basic Medical Research, 5(2), 124–127. https://doi.org/10.4103/2229-516X.157168

Llamas, L. (s.f.). Qué son y cómo usar los GPIO en Raspberry Pi. Recuperado el 27 de septiembre de 2026, de https://www.luisllamas.es/como-usar-gpio-raspberry-pi/

Newbiely. (s.f.). Raspberry Pi Pico – Button – Debounce. Recuperado el 27 de septiembre de 2026, de https://newbiely.com/tutorials/raspberry-pico/raspberry-pi-pico-button-debounce

Raspberry Pi Ltd. (2025). RP2350 datasheet (Build-version d126e9e-clean, 29 de julio de 2025). https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf

Raspberry Pi Ltd. (2026a). Getting started with Raspberry Pi Pico-series: C/C++ development with Raspberry Pi Pico-series and other Raspberry Pi microcontroller-based boards (Release 24). https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf

Raspberry Pi Ltd. (2026b). Raspberry Pi Pico 2 datasheet: An RP2350-based microcontroller board (Release 5). https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf

Raspberry Pi Ltd. (2026c). Raspberry Pi Pico-series C/C++ SDK: Libraries and tools for C/C++ development on Raspberry Pi microcontrollers (Build-version ea68919-clean, 10 de septiembre de 2026). https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf

Texas Instruments. (s. f.). Motor drivers. https://www.ti.com/product-category/motor-drivers/overview.html

Texas Instruments. (s. f.). Mitigating EMI and managing thermals in motor drivers. https://www.ti.com/document-viewer/lit/html/SSZT236

maxon. (s. f.). PWM power stage: Current ripple and motor chokes. maxon Support. https://support.maxongroup.com/hc/en-us/articles/360005046213-PWM-power-stage - Current-ripple-Motor-chokes

Dietrich, S. (2022, 23 de marzo). *Understanding the basics of pulse width modulation (PWM)* [Comprender los fundamentos de la modulación por ancho de pulso (PWM)]. Control.com. https://control.com/technical-articles/understanding-the-basicsof-pulsewidth-modulation-pwm/

Sufyan, M. (2025, 9 de septiembre). *What is a PWM signal? Fundamentals and practical applications for engineers* [¿Qué es una señal PWM? Fundamentos y aplicaciones prácticas para ingenieros]. Wevolver. https://www.wevolver.com/article/ what-is-a-pwm-signal-fundamentals-and-practical-applications-for-engineers